

On the Correctness of Ukkonen's Algorithm Without Suffix Links

A Practical Commentary on Suffix Tree Construction

Advanced Algorithms Assignment 1

Nicholas.S

October 2, 2026

Abstract

In this document, a naive implementation for Ukkonen's algorithm without suffix links has a worst case running time of $O(n^2)$ for suffix tree construction. The algorithm follows three extension rules, and must traverse down edges individually to create a split. It becomes evident, through a step-by-step visualised walk-through, that the suffix link implementation is far better optimised, running in linear time relative to the size of the input string. Additionally, the implementation code is provided with an in-depth correctness commentary on how each step contributes to running each extension rule, necessary pre and post-conditions, effect on the loop invariant and the importance of each step.

Keywords: Ukkonen's algorithm, suffix trees, suffix links, correctness commentary, string, lemma, proof

Contents

1	Introduction	3
1.1	The Three Rules of Ukkonen's	3
2	Code Implementation in C++	4
2.1	Brief Explanation	4
2.2	Header File	4
2.3	Source File	4
3	A Worked Example	7
3.1	Algorithm Walk-through	7
3.2	Where Suffix Links Differ	12
4	The Correctness of The Naive Ukkonen's Algorithm	14
4.1	Initialisation	14
4.2	The Main Loop	14
4.3	Searching For a Node	15
4.4	Equality of Characters	16
4.4.1	Case 1: Length is outside the edge	16
4.4.2	Case 2: Splitting the Edge	16
4.4.3	Case 3: Skipping Operation	17
4.5	Conclusion	17
5	Running the Algorithm	17
5.1	Test Run Results	17
6	Use of Generative AI	19
	References	21

1 Introduction

Ukkonen's algorithm is uniquely known for constructing a suffix tree where the input string is read from left-to-right. It was formalised by Esko Ukkonen in "On-line construction of suffix trees" [1], and at any step of the construction, there is a valid implicit suffix tree. Suffix trees were introduced in "Linear Pattern Matching Algorithms" [2], as a way to reduce basic string operations in reduced time from standard algorithms.

My own implementation of Ukkonen's is a naive approach, where each edge is traversed manually from the root on each iteration, whenever the edge is reset. Whilst it still returns a valid and fully constructed suffix tree of any string which doesn't contain the character '\$', the worst case upper bound is $O(n^2)$ where n is the length of the string. The main advantage of my implementation, however, is that there is some form of reduced space consumption because there is no stored suffix link.

1.1 The Three Rules of Ukkonen's

To understand Ukkonen's algorithm, there are three essential rules to how it exactly works. Let there exist string s such that $s = s_0, s_1 \dots s_n$. Then, assume we have already some online, partially constructed suffix tree of s , denoted as T . Then, let us use i and j as indices such that $0 \leq i \leq j \leq n+1$, where $j+1$ represents the current character being processed.

- (Rule.1) If the path from $s[i \dots j]$ ends on a leaf edge, then $s[j+1]$ is inserted such that the path is now $s[i \dots j+1]$ [4, p. 9, l. 6].
- (Rule.2) If the path from $s[i \dots j]$ ends on a non-leaf edge, where $0 \leq k \leq j$, then remain on the edge until some $j+k$, such that $s[i \dots j+k] = s[j+k]$. At this point, split the existing edge from $(i \dots i+k)$, with two child nodes containing edges $(j+k, j+k)$, and $(i+k, j+k)$ [4, p. 9, l. 7].
- (Rule.3) Otherwise, $s[i+k]$ moves on without insertion [4, p. 9, l. 8].

2 Code Implementation in C++

2.1 Brief Explanation

The following code was my own hand-written implementation of the naive Ukkonen's without suffix links. The code contains a header file with the necessary structs to implement the algorithm. This file is imported into the source file, which implements the *create* function to return the root node of a constructed suffix tree based on the given word.

There is no ability to search, the only example of it running is against JSON test cases with my own implementation of the test runner.

It's quite clear that the code does not represent the initial pseudocode provided in 'online construction' page 12. This was an intentional move to try to derive the code through only a theoretical understanding.

2.2 Header File

```
1  src/headers/suffix_tree.hpp
2  #ifndef SUFFIX_TREE_HPP
3  #define SUFFIX_TREE_HPP
4
5  #include <string>
6  #include <memory>
7  #include <unordered_map>
8
9  namespace Suffix_Tree {
10     // Simply contains the indices of a substring [a,b] of word. This is far more efficient than an array.
11     struct Ind {
12         int a;
13
14         // Allows for really simple indexing, the algorithm can simply increase the value on the heap and all leaf nodes can
15         // point to the same int.
16         std::shared_ptr<int> b;
17     };
18
19     // A struct to contain all data of the nodes in a suffix tree. A tree doesn't exist only its nodes.
20     struct Node {
21         // Each node contains a unique suffix except for the root, these are represented as indices.
22         std::unique_ptr<Ind> Indices;
23
24         // Can check if a character is already exists and returns it's node in constant time.
25         std::unordered_map<char, std::unique_ptr<Node>> Children;
26     };
27
28     // Contains necessary positional data on where in the tree is currently being processed
29     struct Active_Point {
30         // The parent of the current edge which is being operated on
31         Node* Parent;
32
33         // The incoming edge to a node, which is also a child edge of the parent node
34         Node* Edge;
35
36         // How far into the active edge is being processed from index a
37         int Length;
38     };
39
40     // Safely returns the character at an index and $ at size
41     char get_char(const std::string& w, int i);
42
43     // Returns a pointer to the root node of a suffix tree. Using the root node traversal is possible.
44     std::unique_ptr<Node> create(const std::string& word);
45 }
46
47 #endif
```

2.3 Source File

```
1  src/suffix_tree.cpp
2
3  #include "suffix_tree.hpp"
4
5  namespace Suffix_Tree {
6     // Returns word[i] of index otherwise $ for terminal processing
7     char get_char(const std::string& w, int i) {
8         if (i != w.size()) return w[i];
9     }
10 }
```

```

7     else return '$';
8 }
9
10 // The implementation of Ukkonen's algorithm which constructs a suffix tree
11 // This is without suffix links, with a worst case upper bound of  $O(n^2)$ 
12 std::unique_ptr<Node> create(const std::string& word) {
13     // Root node has no character and no incoming edge
14     std::unique_ptr<Node> root_node = std::make_unique<Node>(nullptr);
15
16     // Keeps track of current parent node, edge and index of comparison with length -- all 3 crucial to Ukkonen's
17     Active_Point active;
18
19     // The algorithm begins by operating on the root node
20     active.Parent = root_node.get();
21
22     // No outgoing edges exist yet to process
23     active.Edge = nullptr;
24     active.Length = 0;
25
26     // The number of leftover suffixes skipped to be inserted later
27     int remainder = 0;
28
29     // EXTENSION RULE 1 -> Automatically updates word[a, i] such that the leaf edge extends each iteration.
30     std::shared_ptr<int> i = std::make_shared<int>(0);
31
32     // We only need to loop while there are suffixes to add.
33     while (*i <= word.size()) {
34
35         // First find a candidate edge with the same prefix
36         std::unordered_map<char, std::unique_ptr<Node>>::const_iterator find_edge =
37             active.Parent->Children.find(get_char(word, *i - active.Length));
38
39         // EXTENSION RULE 2.i -> When there is no such child node of word[i] at active parent, then create a new node and
40         // add
41         if (find_edge == active.Parent->Children.end()) {
42
43             // Indices are from current character to index
44             std::unique_ptr<Ind> ind = std::make_unique<Ind>(*i, i);
45
46             // Creating the new child node
47             active.Parent->Children[get_char(word, *i - active.Length)] = std::make_unique<Node>(std::move(ind));
48
49             // Suffix inserted so one less total suffixes remain, otherwise move to next index
50             if (remainder > 0) --remainder;
51             else ++i;
52
53             // Starts from the top every time or else would be creating non-existent suffixes
54             active.Parent = root_node.get();
55             active.Edge = nullptr;
56
57             // Current suffix has been inserted, move to next with reset length
58             active.Length = remainder;
59
60             continue;
61         }
62
63         // The found edge is now traversed down because of equal prefixes
64         active.Edge = find_edge->second.get();
65
66         // There must always be a character to process.
67         // If it's too large, step down to the next node and continue
68         if (active.Length >= *active.Edge->Indices->b - active.Edge->Indices->a) {
69             active.Parent = active.Edge;
70             active.Length -= *active.Edge->Indices->b - active.Edge->Indices->a;
71             active.Edge = nullptr;
72         }
73
74         // EXTENSION RULE 2.ii -> word[a, b) exists such that word[b] != word[*i], and therefore the edge must now be
75         // split
76         // word[a, a + l), where l is the current active length, with 2 children of n1[*i, i) and n2[b, *i).
77
78         // There is an edge which exists, we must traverse it and split at the unequal index
79         else if (get_char(word, active.Edge->Indices->a + active.Length) != get_char(word, *i)) {
80
81             // This is the new suffix added
82             std::unique_ptr<Ind> ind = std::make_unique<Ind>(*i, i);
83
84             // This is the existing suffix bottom half being split
85             std::unique_ptr<Ind> ind2 = std::make_unique<Ind>(active.Edge->Indices->a + active.Length,
86                 active.Edge->Indices->b);
87
88             // Update the active node edge to only be the active length from a
89             active.Edge->Indices->b = std::make_shared<int>(active.Edge->Indices->a + active.Length);

```

```

86
87 // Create split edge from original edge
88 std::unique_ptr<Node> bottom_edge = std::make_unique<Node>(std::move(ind2));
89
90 // Simply move the existing hashmap of children
91 bottom_edge->Children = std::move(active.Edge->Children);
92
93 // Clears the children which are no longer linked to upper half of the old edge
94 active.Edge->Children.clear();
95
96 // Add new nodes as child nodes to split edge
97 active.Edge->Children[get_char(word, *i)] = std::make_unique<Node>(std::move(ind));
98 active.Edge->Children[get_char(word, active.Edge->Indices->a + active.Length)] = std::move(bottom_edge);
99
100 // Suffix inserted so one less total suffixes remain, otherwise move to next index
101 if (remainder > 0) --remainder;
102 else ++i;
103
104 // Reset to root for remainder suffixes to be inserted (just a rerun with 1 less char)
105 active.Parent = root_node.get();
106 active.Edge = nullptr;
107
108 // Current suffix has been inserted, must move onto the next such that length resets
109 active.Length = remainder;
110 }
111
112 // EXTENSION RULE 3 -> word[a, b) exists such that word[b + 1] = word[i]. This means the next char should be
113 ↪ processed so length is bumped.
114 else {
115
116 // A suffix is skipped to be inserted later, so its a new remainder
117 ++remainder;
118
119 // We know for next suffix all letters will equal, bump it so that they only check the newest one
120 ++active.Length;
121
122 ++i;
123 }
124 }
125 return root_node;
126 }
127 }

```

3 A Worked Example

3.1 Algorithm Walk-through

The following is a visualised, step-by-step walk-through of how my implementation constructs 'aabab' into a suffix tree.

Active Point (A), lines 29:38, header. A struct which contains pointers to nodes *parent* and *edge*, and an integer *length*. The *parent*, ρ , is a node which is being searched for an edge, this is used to confirm if a character already exists at this point in the tree. The *edge*, e is a pointer to a node such that (a, b) form the indices of an edge to ρ , $s[a \dots b]$. The *length*, len , variable is used to label how far into e is being processed. For example, $len = 2$ means the comparison is with some $s[a + len]$.

Remainder (R), line 27, source. An integer which denotes the number of suffixes which have been skipped. This is being applied, where the suffixes will eventually be implemented at a later time.

Index (i), line 30, source. A shared pointer which points to how far into the string 'aabab' the algorithm is up to. For example, if $i = 2$, then up to 'aab' is being processed. This is how extension rule one is applied in constant time. Whenever some e contains (a, i) , then whenever i is altered, that same e will instantly update to use the new value. For example, if some node exists with $[0, i]$, then when $i = 0$, the node has indices $[0, 0]$. When $i = 1$, the indices will automatically become $[0, 1]$, that is how the shared pointer works.

$$A = (R, \emptyset, \emptyset) \quad R = \emptyset \quad i = 0$$



Figure 1: The initial state, with only the root.

1. Initialisation, lines 14:30. The suffix tree root node **R** has been initialised, where the root has no incoming edge, and no children. Also, $\rho = \mathbf{R}$.

2. Create a New Node, lines 36:59.

$$A = (R, \emptyset, \emptyset) \quad R = \emptyset \quad i = 1$$



Figure 2: A new leaf for a under the root.

The loop begins. i begins as 0, such that the word being processed is $s[0]$ which is 'a'. Then, ρ is searched to see whether it contains a child node that begins with 'a'. Here, this is false, and so the new node is created from $[0, i]$. This is a case of (Rule.2), where $\epsilon \neq 'a'$, and so the node is created. Lastly, i is increased by one, to mark the end of (Rule.2) so move to the next character.

3. Skipping Operation, *lines 113:122.*

$$A = (R, 'a', 1) \quad R = 1 \quad i = 2$$



Figure 3: Rule 3 defers the suffix **a**.

At $i = 1$, $s[i] = 'a'$. Given ρ contains a child node beginning with 'a', e points to that node instead. At this point, $len = 0$, and so this becomes $s[0] = s[1]$, where now (Rule.3) is applied, and nothing is done. Then, $len = 1$ as the first characters have been compared, $R = 1$ to allow the current suffix to be inserted at a later time, and $i = 2$.

4. Split Current Edge and Create New Nodes, *lines 76:110.*

$$A = (R, \emptyset, 0) \quad R = 0 \quad i = 2$$

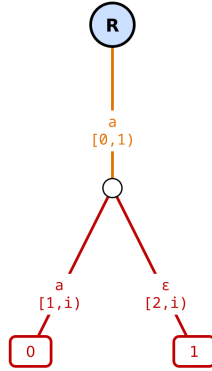


Figure 4: The leaf **aa** is split after **a**.

The only leaf edge in the tree node contains indices from $(0, 2)$ now. The current character, $s[i]$ is 'b'. Because from path $s[0+1] = s[1]$, there exists $s[2]$ after such that $s[i] \neq s[\alpha + len]$, where α is the first index in e . According to (Rule.2), the edge must now be split such that e becomes $(0, 1)$, then e_0 is created with $(2, 2)$, and e_1 is created with $(1, 2)$ as child nodes.

5. Add New Node.

$$A = (R, \emptyset, 0) \quad R = 0 \quad i = 3$$

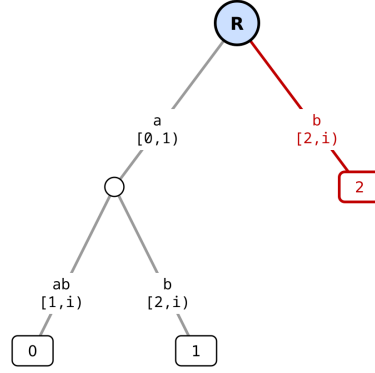


Figure 5: A new leaf for **b** under the root.

The split in the previous step inserted the suffix 'ab', so $\mathbf{R} = 0$ and $\mathbf{i}$ stays at 2 to insert the final suffix 'b'. With $len = 0$, ρ is searched for a child beginning with $s[\mathbf{i}] = \text{'b'}$. None exists, so (Rule.2) creates a new node from $(2, \mathbf{i})$. Because $\mathbf{R} = 0$, $\mathbf{i}$ is increased to 3.

6. Search For Child Node.

$$A = (R, \text{'a'}, 1) \quad R = 1 \quad i = 4$$

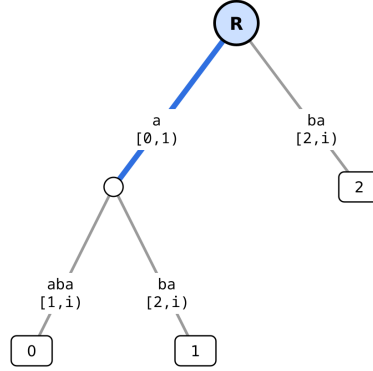


Figure 6: Rule 3 defers the suffix **a** again.

At $\mathbf{i} = 3$, $s[\mathbf{i}] = \text{'a'}$. ρ contains a child beginning with 'a', so e points to the internal node with indices $(0, 1)$. As $len = 0$, the comparison is $s[0] = s[3]$, which are both 'a', so (Rule.3) is applied and nothing is done. Then $len = 1$, $\mathbf{R} = 1$ and $\mathbf{i} = 4$.

7. Edge Traversal.

$$A = ('a', \emptyset, \emptyset) \quad R = 1 \quad i = 4$$

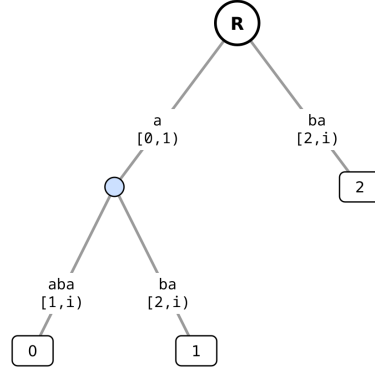


Figure 7: The active point walks down to the internal node **a**.

At $i = 4$, e is chosen by $s[i - len] = s[3] = 'a'$, which is the internal node with indices $(0, 1)$. Its edge has a length of 1, and $len = 1$ has reached its end, so there is no character left on e to compare. Lines 66:70 traverse down the edge: ρ becomes the internal node, len is reduced by the edge length to 0, and e is cleared. Nothing is inserted, so R and i are unchanged.

8. Skipping Operation.

$$A = ('a', 'b', 1) \quad R = 2 \quad i = 5$$

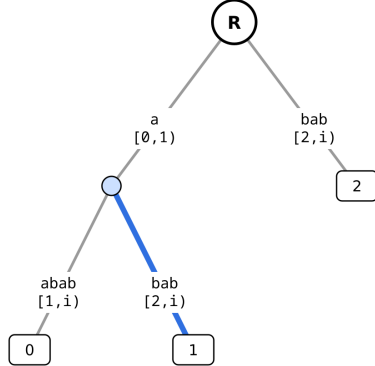


Figure 8: Rule 3 below the internal node, with two suffixes pending.

Still at $i = 4$, ρ is now the internal node, and with $len = 0$ it is searched for a child beginning with $s[4] = 'b'$. That child is the leaf $(2, i)$, and $s[2] = s[4] = 'b'$, so (Rule.3) applies again. Then $len = 1$, $R = 2$ and $i = 5$, so two suffixes, 'ab' and 'b', are now waiting to be inserted.

9. Split Current Edge.

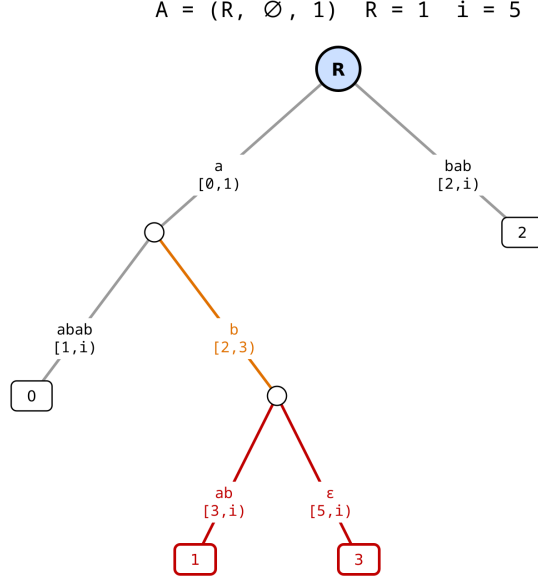


Figure 9: The leaf **bab** below the internal node is split after **b**.

At $i = 5$, the terminal character $\$$ is processed. From the internal node, e is the leaf $(2, i)$, which spells 'bab'. The next character on e is $s[2 + len] = s[3] = 'a'$, which is not $\$$, so (Rule.2) splits e into $(2, 3)$, with a new node $(5, i)$ for $\$$ and a node $(3, i)$ for the rest of the old edge. This inserts the suffix 'ab\$'. R is reduced to 1, so i stays at 5, and the active point resets to the root, with len set to R , which is 1.

10. Split Current Edge.

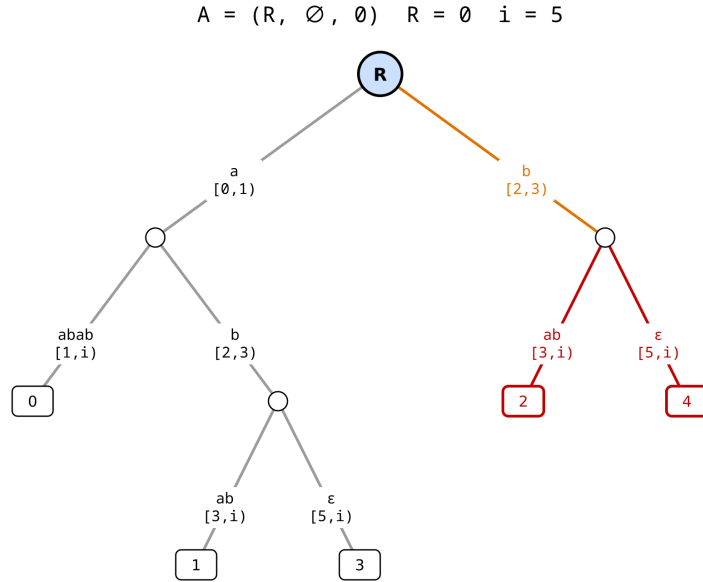


Figure 10: The root's leaf **bab** is split after **b**.

From the root, e is chosen by $s[i - len] = s[4] = 'b'$, which is the root's leaf $(2, i)$, also spelling 'bab'. Again, $s[2 + len] = s[3] = 'a'$ is not $\$$, so this edge is split in the same way, inserting the suffix 'b\$'. R is reduced to 0, so i stays at 5 and $len = 0$.

11. Add New Node.

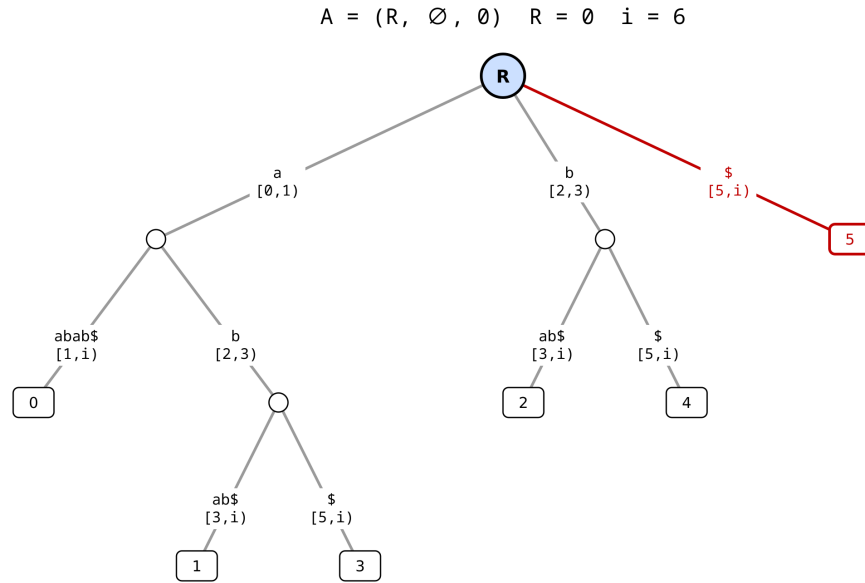


Figure 11: The completed suffix tree of `aabab$`.

Only the suffix '\$' remains. The root has no child beginning with \$, so (Rule.2) creates a new node from $(5, i)$. As $R = 0$, i is increased to 6, which is past the end of 'aabab\$', so the loop ends. The completed tree has six leaves, one for each suffix of 'aabab\$'.

3.2 Where Suffix Links Differ

Between Steps 4 and 5.

$$A = (R, \emptyset, \emptyset) \quad R = 1 \quad i = 2$$

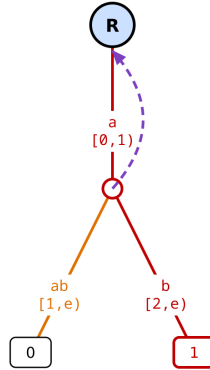


Figure 12: The new internal node `a` is given a suffix link to the root.

In the suffix link version, the split at $i = 2$ inserts a new internal node from $(0, 1)$ above the old leaf, rather than shortening the old node in place, so the old leaf's edge now begins at index 1. The new internal node spells 'a', and removing its first character leaves the empty string, so its suffix link points to the root. As ρ is the root, len is reduced by one to 0, rather than being set to R . $R = 1$ here, not 0, because this version also counts the current character's suffix, 'b', as pending.

Between Steps 9 and 10.

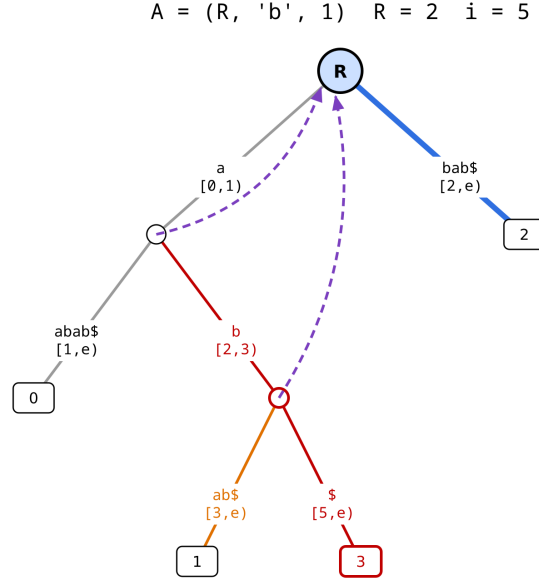


Figure 13: The active point follows the suffix link of **a** to the root, instead of resetting.

At $i = 5$, the leaf below the internal node 'a' is split after 'b', creating the internal node 'ab', which is given a suffix link to the root for now. In my implementation, the active point then resets to the root with len set to **R**. Here, ρ is not the root, so the active point follows the suffix link of 'a' instead, keeping e and $len = 1$. The link of 'a' points to the root, so both versions reach the same position, but when a link points to another internal node, following it avoids traversing down again from the root.

Between Steps 10 and 11.

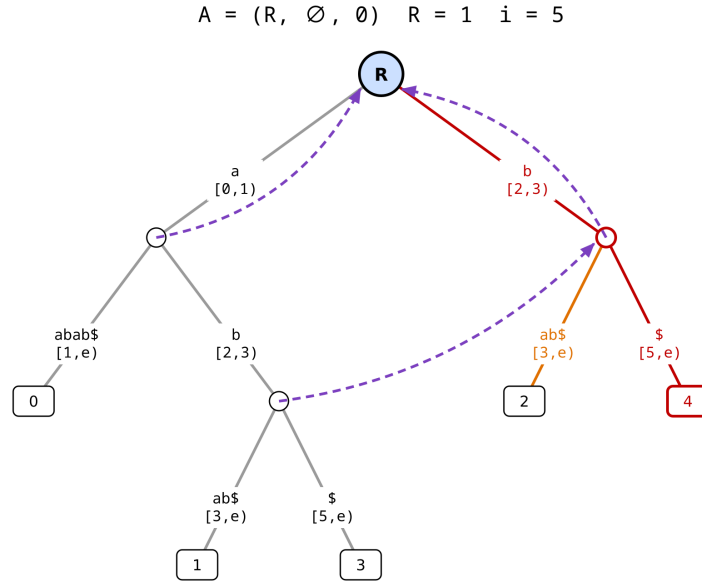


Figure 14: **ab** now links to **b**, and **b** links to the root.

From the root, the leaf 'bab\$' is split after 'b', creating the internal node 'b'. As 'ab' was the last internal

node created in this phase, its suffix link is set to 'b', since removing the first character of 'ab' leaves 'b'. The new node 'b' links to the root. As ρ is the root, len is reduced by one to 0, and the next step adds the \$ leaf as in my implementation.

4 The Correctness of The Naive Ukkonen's Algorithm

Precondition: Let Σ exist as an alphabet such that $\$ \notin \Sigma$, and where L exists that $L \subseteq \Sigma^*$. There must be some string s , such that $s \in L$, also represented as $s = s_0, s_1 \dots s_{n-1}$.

Post-condition: (P0.) Given string s , where $\sigma(s)$ constructs a suffix tree of s . It is, then, expected that:

- (P0.1) the root node, R , exists such that $\sigma(s) = R$
- (P0.2) R contains at least one node for the terminal \$, ν_n and where $\eta_R = \{\nu_0 \dots \nu_n\}$.
- (P0.3) where $\alpha(\nu)$ is the inclusive beginning and $\beta(\nu)$ is the exclusive ending of ν , $\forall \nu \in \eta_R, 0 \leq \alpha(\nu) \leq \beta(\nu) \leq n + 2$.
- (P0.4) all child nodes of R must have a unique beginning index, $\forall i, j \in \{0 \dots n\}, i \neq j : \alpha(\nu_i) \neq \alpha(\nu_j)$.
- (P0.5) where R is the root of suffix tree T_R , $\forall \nu \in \eta_R, \{\exists T_\nu | \nu \neq \nu_n\}$.
- (P0.6) it follows, $\forall \nu' \in \eta_R, \{\exists T_{\nu'} | \nu' \neq \nu_{|\eta_{\nu'}|}\}$.
- (P0.7) any possible sub-string $s[i, j]$ where $0 \leq i \leq j \leq n$, can be searched within T_R .

4.1 Initialisation

Post-condition: (P1.) Following initialisation, it is expected that:

- (P1.1) R exists such that $\alpha(R) = \beta(R) = \perp$.
- (P1.2) the active point, AP , contains parent node ρ , such that $\rho = R$.
- (P1.3) there is some global index i , such that any node ν where $\beta(\nu) = i$, then on $i = i + 1$, $\beta(\nu) = i + 1$.
- (P1.4) there is also i' where instead $\beta(\nu) = i'$ when $i = i + 1$.

4.2 The Main Loop

Post-condition: (P2) given the main loop is the entirety of the algorithm outside of initialisation, then (P2) holds the same post-conditions as (P0).

Loop Invariant: (I)

- (I.1) That $0 \leq i \leq n + 2$.
- (I.2) $\alpha(R) = \beta(R) = \perp$.
- (I.3) Where rem denotes the remainder, then $0 \leq rem \leq n$.
- (I.4) Where len denotes the length in AP, then $0 \leq len \leq n + 1$.
- (I.5) There is $\tau_{(i, rem)}$, a partially constructed suffix tree at operation (i, rem) , such that $\rho \in N\tau_{(i, rem)}$. Allow $N\tau_{(i, rem)}$ to be the set of all nodes in $\tau_{(i, rem)}$.

Loop Variant: The loop terminates for the following $(n + 1 - i, rem, len)$. $n + 1 - i$, .

1. $n + 1 - i$ allows the loop to access all characters in s , as well as the $\$$ before reaching 0
2. rem exists such that it tracks the number of remaining suffixes yet to be placed into the tree. All suffixes of s must be placed into the tree, thus rem must be 0 for the loop to terminate.
3. len tracks how far into an edge is currently being compared. Each edge split will reduce the length to rem , and so for the loop to terminate len must be 0.

4.3 Searching For a Node

Post-condition: (P3) Either a node, ν is found such that $\nu \in N_{\tau(i, rem)}$ (Case 2), or the node doesn't exist (Case 1). All of **(I)** holds.

Lemma 4.1 *The only node which ever is searched for any child node, is ρ .*

Proof of Lemma 4.1. *There is only one occurrence of searching for a child node, on **line 36**. The exact node being searched against is ρ .*

Case 1: no such child exists.

Post-condition (P3.a) .

(P3.a.1) A new node, ν is created such that $\alpha(\nu) = i' - len$.

(P3.a.2) $\nu \in \eta_\rho$.

Lemma 4.2 *Where no node was initially found, the creation of any node ν at operation (i, rem) , must only become a child of ρ , $\eta_\rho = \eta_\rho \cup \nu$.*

Proof of Lemma 4.2. *On **line 42**, the node ν is created, where only **line 45** ν becomes a child of ρ . The only other occurrence of insertion is on **lines 97, 98**, where the edge is split.*

Lemma 4.3 *Where a node was not found, creating a child node ends the operation.*

Proof of Lemma 4.3. *On **line 58**, the clause which allows for the creation of a node skips and moves to either operation $(i + 1, 0)$ or $(i, rem - 1)$.*

The loop is re-iterated. A new node has been created, and (Rule.2) is complete.

Importance. Creating a new node, ν , at (i, rem) allows for the addition of new suffixes of s .

Omission means, initially on operation $(0, 0)$, when $\rho = R$, then $s[i' - len]$ can never create a new node, preventing (P0.2). All of **(I)** would hold.

Case 2: Use an existing node.

Post-condition: (P3.b). The edge node in AP , $e = \forall \nu' \in \eta_\rho, \{\exists s[\alpha(\nu')] = s[i' - len]\}$.

Lemma 4.4 *Before the edge is not split, there cannot be any two nodes, which their first indices index the same character in s on ρ , $\forall \nu' \in \eta_\rho, \{\exists s[\alpha(\nu')] = s[i' - len]\}$.*

Proof of Lemma 4.4. *All edge split logic occurs on **lines 97, 98**. By **Lemma 4.1** and **Lemma 4.2**, a node can only be added if it's not found. Because the node is found, it cannot be added, and therefore there can never have nodes such that any two beginning indices point to the same character in s . In order for **(P3)** to hold, then a node must be selected, which already exists, fulfilling **(P3.b)**.*

Importance. There must be an edge so that either (Rule.2) or (Rule.3) may be applied. All of **I** still holds.

Omission means, the edge can never be split at an operation. Therefore, **(P0.4)** will always fail.

4.4 Equality of Characters

4.4.1 Case 1: Length is outside the edge

Post-condition: (P5.a).

(P5.a.1) The value of e moves to ρ .

(P5.a.2) $len < \beta(\rho) - \alpha(\rho)$

(P5.a.3) $e = \perp$.

Lemma 4.5 *For any character comparisons, $len < \beta(e) - \alpha(e)$.*

Proof of Lemma 4.5. *Lines 66, 76 and 113 all exist exclusively such that only one can be done in each iteration. Therefore, whenever $len \geq \beta(e) - \alpha(e)$, the length is updated to fit within a new edge.*

Importance. There must be a valid character comparison so (Rule.2) or (Rule.3) may be applied. All of **I** still holds.

Omission means, any edge e can allow comparisons of characters greater than $\beta(e)$, introducing suffixes which do not exist, and breaking **(P0.7)**.

4.4.2 Case 2: Splitting the Edge

Post-condition: (P5.b).

(P5.b.1) $\beta(e) = i' - len$.

(P5.b.2) allow $\tau_{s[i'-len],(i,rem)}$ to be the partially constructed suffix tree, where, e exists as the root node. Then, $|\eta_{\tau_{s[i'-len],(i,rem)}}| = 2$.

(P5.b.3) The first node ν_0 in $\eta_{\tau_{s[i'-len],(i,rem)}}$ exists such that $\alpha(\nu_0) = i'$ and $\beta(\nu_0) = i$.

(P5.b.4) The second node ν_1 also exists, such that $\alpha(\nu_1) = i' - len$, and $\beta(\nu_1) = i$.

Creating New Nodes

(P5.b.1) There exist two nodes, ν_0 and ν_1 without any incoming edges or children.

(P5.b.2) ν_0 exists such that $\alpha(\nu_0) = i'$ and $\beta(\nu_0) = i$.

(P5.b.3) ν_1 also exists, such that $\alpha(\nu_1) = i' - len$, and $\beta(\nu_1) = i$.

Lemma 4.6 *The first new node, ν_0 , exists such that $\alpha(\nu_0) = i'$ and $\beta(\nu_0) = i$.*

Proof of Lemma 4.6. *The only point where a first node is created is on **line 97**, which uses indices from **line 79**. The indices are (i', i) .*

Lemma 4.7 *The second new node, ν_1 , exists such that $\alpha(\nu_1) = i' - len$ and $\beta(\nu_1) = i$.*

Proof of Lemma 4.7. *The only point where a second node is created is on **line 88**, which uses indices from **line 82**. The indices are $(i' - len, i)$.*

importance. A split cannot occur without two new nodes. *Omission means,* the split simply cannot occur, and the tree becomes an implicit suffix tree.

Transferring of Child Nodes

Post-condition: (P5.c).

(P5.c.1) ν_1 contains old children

(P5.c.2) e contains no children

Lemma 4.8 *When an edge is split, all of children in e must be moved to ν_1 .*

Proof of Lemma 4.8. *The only occurrence of any children being moved is on **line 91**. This explicitly sets ν_1 as to retrieve the children of e . Then, on **line 94**, all of the children which existed in e are cleared in memory.*

importance. In order for an edge to be split, the children of e must be moved to ν_1 . *Omission means*, that e will allow for the searching of characters which do not occur after it, thus breaking **(P0.7)**.

Adding the New Nodes

(P5.d.1) $|\tau_{s[i-len],(i,rem)}| = 2$

Lemma 4.9 *Only two nodes can be created from a split such that they join e .*

Importance. The edge is split, so the nodes must be added. *Omission means* There is no way to add suffixes which share a common beginning unless its the largest suffix.

4.4.3 Case 3: Skipping Operation

Post-condition: (P6) Same as (P0). Loop invariant holds.

Lemma 4.10 *Characters can be equal.*

Importance. When two indices are equal this does not mean it should split the edge. *Omission means* New suffixes will be added which are not part of the string.

4.5 Conclusion

The algorithm meets all the necessary lemmas such that the root node will return the root of a constructed suffix tree without suffix links. As demonstrated, all of Ukkonen's extension rules are complete.

5 Running the Algorithm

The algorithm itself exists solely and entirely as a raw implementation. There is no other piece of code which uses it, outside of the custom test runner, which was built to run JSON test cases against the algorithm.

5.1 Test Run Results

Considerations:

1. These were run locally on a PC once, all sequentially in the same process.
2. Running time is calculated using the `<chrono>` library in `C++`, with *time after execution - time before execution*.
3. Whilst inflated, it proves in the very least that the running time is at least as fast as the provided values.

Observations:

1. The empty string, ϵ has the shortest running time.
2. The longest string 'abcdefabxybcdmabcdex' has the longest running time.
3. There is some correlation such that the larger the word size, the longer the running time.
4. There is a lack of correlation between running time and alphabet size.

Table 1: Suffix tree construction results.

Word	Passed	Word Size	Alphabet Size	Total Nodes	Time (ms)	< 30 μ s
ε	Yes	0	0	2	0.0009	Yes
a	Yes	1	1	3	0.0014	Yes
ab	Yes	2	2	4	0.0021	Yes
aa	Yes	2	1	5	0.0032	Yes
abc	Yes	3	3	5	0.0030	Yes
aab	Yes	3	2	6	0.0040	Yes
aba	Yes	3	2	6	0.0041	Yes
aaa	Yes	3	1	7	0.0050	Yes
aaab	Yes	4	2	8	0.0056	Yes
abab	Yes	4	2	8	0.0055	Yes
baaa	Yes	4	2	8	0.0063	Yes
aaaa	Yes	4	1	9	0.0065	Yes
xabxac	Yes	6	4	10	0.0067	Yes
abcbabc	Yes	6	3	11	0.0080	Yes
banana	Yes	6	3	11	0.0138	Yes
abacaba	Yes	7	3	12	0.0089	Yes
cdddcdc	Yes	7	2	14	0.0109	Yes
xyzxyxyz	Yes	9	4	16	0.0123	Yes
abcbabcabc	Yes	9	3	17	0.0130	Yes
abcbababcd	Yes	10	5	17	0.0125	Yes
mississippi	Yes	11	4	19	0.0148	Yes
dedododeeodo	Yes	12	3	22	0.0180	Yes
abaababaabaab	Yes	13	2	26	0.0245	Yes
abcdefabxybcdmabcdex	Yes	21	10	34	0.0267	Yes

6 Use of Generative AI

Models Used: Claude Opus 5.5, via Claude Code (Anthropic)

Date of Usage: 20/09/26 to 02/10/26

Table 2: Use of generative AI in this paper.

Area	What I Did	What AI Did	Purpose
Formatting	Wrote the document structure and content.	Set up the code listings, figure placement and table styling.	The document compiles cleanly and reads well.
Diagrams	Chose aabab as the example and specified the layout.	Generated the step images and the suffix link images. Red marks new nodes, orange a shortened edge, the shaded node the active parent and the thick blue edge the active edge.	Each step of the walk-through is shown visually.
Step 1 (Fig. 1)	Wrote the step description.	Generated the image of the root alone, with $A = (R, \emptyset, 0)$, $R = 0$ and $i = 0$ above it.	Shows the state after initialisation.
Step 2 (Fig. 2)	Wrote the step description.	Generated the image of the new leaf for a under the root, drawn in red, with i advanced to 1.	Shows a new node being created.
Step 3 (Fig. 3)	Wrote the step description.	Generated the image of rule 3, with the active edge a in blue, $A = (R, a, 1)$ and $R = 1$.	Shows a suffix being deferred.
Step 4 (Fig. 4)	Wrote the step description.	Generated the image of the leaf aa split after a , with the shortened edge in orange and the new internal node and leaf in red.	Shows an edge being split.
Step 5 (Fig. 5)	—	Generated the image of the new leaf for b under the root, with i advanced to 3.	Shows a new node added at the root.
Step 6 (Fig. 6)	—	Generated the image of rule 3 on the internal edge a , with $R = 1$ and a length of 1.	Shows a deferral along an internal edge.
Step 7 (Fig. 7)	—	Generated the image of the walk-down, with the internal node a shaded as the new parent and a length of 0.	Shows edge traversal.
Step 8 (Fig. 8)	—	Generated the image of rule 3 below the internal node, with $A = (a, b, 1)$ and $R = 2$.	Shows two suffixes pending.
Step 9 (Fig. 9)	—	Generated the image of the leaf bab below the internal node split after b , with $A = (R, \emptyset, 1)$ and $R = 1$.	Shows a split below an internal node.
Step 10 (Fig. 10)	—	Generated the image of the root's leaf bab split after b , with $R = 0$.	Shows the next pending suffix inserted.
Step 11 (Fig. 11)	—	Generated the image of the new $\$$ leaf and the completed tree, with six leaves numbered 0 to 5.	Shows the final suffix tree.
Step Explanations	Wrote the explanations for steps 1 to 4.	Wrote the explanations for steps 5 to 11, following my notation.	Every step of the walk-through is explained.
Suffix Link Explanations	Chose the format for where the two versions differ.	Wrote the explanations for the three suffix link figures.	The differences from suffix links are explained.
Captions and Headings	Named the steps.	Wrote the figure captions and renamed some step headings.	Each figure matches its step.
Results Table	Built the test runner and its timing.	Filled the table from a test run and added the pass and timing columns.	The results are reported accurately.
Introduction	Wrote the introduction.	Reviewed it for errors and suggested citations.	The background is accurate.
Invariants	Worked out the invariants on paper.	Clarified the concepts with examples and checked my reasoning.	The correctness argument is sound.
Loop Variant	Wrote the loop variant.	Reviewed it and tested it on an instrumented copy of the code, then explained what was missing from the argument.	The termination argument can be checked.
Complexity Analysis	Wrote the complexity analysis, then removed it.	Reviewed it, counted loop iterations on test strings, and changed the running time claim in the abstract.	The abstract matches the paper.
References	Found the Duke notes and chose the page and line numbers.	Formatted the citations and the bibliography entry in IEEE style.	Sources are cited correctly.
Final Review	Asked for a review before submission.	Listed missing explanations, incorrect notation, sections to split and missing parts.	Remaining issues are known.
Proofreading	Approved the list of quick fixes.	Fixed spelling and notation errors, removed an empty subsection, and labelled each proof with its lemma number.	The document reads correctly.

References

- [1] E. Ukkonen, “On-line construction of suffix trees,” *Algorithmica*, vol. 14, no. 3, pp. 249–260, Sep. 1995, doi: 10.1007/BF01206331.
- [2] P. Weiner, “Linear pattern matching algorithms,” in *Proc. 14th Annu. Symp. Switching and Automata Theory (SWAT 1973)*, Oct. 1973, pp. 1–11, doi: 10.1109/SWAT.1973.13.
- [3] D. Gusfield, *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*. Cambridge, U.K.: Cambridge Univ. Press, 1997, doi: 10.1017/CBO9780511574931.
- [4] P. Gómez, “Suffix trees and its construction,” ch. 5, pp. 49–71, course notes, COMPSCI 260, Duke Univ., Durham, NC, USA, Fall 2014. [Online]. Available: <https://courses.cs.duke.edu/fall14/compsci260/resources/suffix.trees.in.detail.pdf> (accessed Oct. 2, 2026).